SC2006 – Software Engineering

Lab 4 Test Cases

Lab Group	TCE2
Team	HabitatHero (Group 2)
Members	NG CAI XIAN
	CHONG XIN LE
	CHOW ZEN TENG
	SIM YONG EN, IVAN
	YEONG JIA WEN, JONATHAN
	MUHAMMAD IZZUDDIN BIN HUSEIN

Table of Contents

1. Black Box Testing 3
1.1 Black Box Testing 1 (Boundary Value Analysis) 3
1.2 Black Box Testing 2 (Equivalence Class Combinations) 4
2. White Box Testing 4
2.1 White Box Testing 1 4
2.1.1 Control Flow Graph 4
2.2 White Box Testing 2 6
2.2.1 Control Flow Graphs 6

1. Black Box Testing

We will be testing on the control class RecommendationEngine.java as it is the core of our application and hence the most vital part of our project.

1.1 Black Box Testing 1 (Boundary Value Analysis)

The Parameters:

- QUIET_NOISE_DB = 55.0
- LOUD_NOISE_DB = 85.0

The Equivalence Classes (ECs):

- EC1 (Quiet): noiseDb <= 55.0 (Should return 1.0)
- EC2 (Moderate): 55.0 < noiseDb < 85.0 (Should return a normalized sliding scale)
- EC3 (Loud): noiseDb >= 85.0 (Should return 0.0)

Table 1: Black Box Testing 1

Test Case	Test Input (noiseDb)	Expected Output	Actual Output

BV1 (Just-below quiet)	54.9	1.0	`Test: BV1 (Just-below quiet)`
			`- Input: 54.9`
			`- Expected: 1.0`
			`- Actual: 1.0`

BV2 (On-boundary quiet)	55.0	1.0	`Test: BV2 (On-boundary quiet)`
			`- Input: 55.0`
			`- Expected: 1.0`
			`- Actual: 1.0`

BV3 (Just-above quiet)	55.1	0.996 (approx normalized)	`Test: BV3 (Just-above quiet)`
			`- Input: 55.1`
			`- Expected: ~0.996`
			`- Actual: 0.9966666666666666`

BV4 (Just-below loud)	84.9	0.003 (approx normalized)	`Test: BV4 (Just-below loud)`
			`- Input: 84.9`
			`- Expected: ~0.003`
			`- Actual: 0.0033333333333331883`

BV5 (On-boundary loud)	85.0	0.0	`Test: BV5 (On-boundary loud)`
			`- Input: 85.0`
			`- Expected: 0.0`
			`- Actual: 0.0`

BV6 (Just-above loud)	85.1	0.0	`Test: BV6 (Just-above loud)`
			`- Input: 85.1`
			`- Expected: 0.0`
			`- Actual: 0.0`

1.2 Black Box Testing 2 (Equivalence Class Combinations)

The Constraints:

- preferredFlatType (Valid: Must exist in HDBDataConstants.VALID_FLAT_TYPES)
- preferredTowns (Valid: Must exist in HDBDataConstants.getAllValidTowns())

Table 2: Black Box Testing 2

Test Case	Test Input (StructuralConstraints object)	Expected Output	Actual Output

EC1: All Valid Inputs	flatType = "4-ROOM", towns = ["BISHAN", "ANG MO KIO"]	Void (Method passes silently without exception)	`Test: EC1 (All Valid Inputs)`
			`- Inputs:`
			`* Flat Type: '4 Room'`
			`* Towns: ['Bishan', 'Ang Mo Kio']`
			`- Expected:`
			`No Exception Thrown`
			`- Actual:`
			`No Exception Thrown`

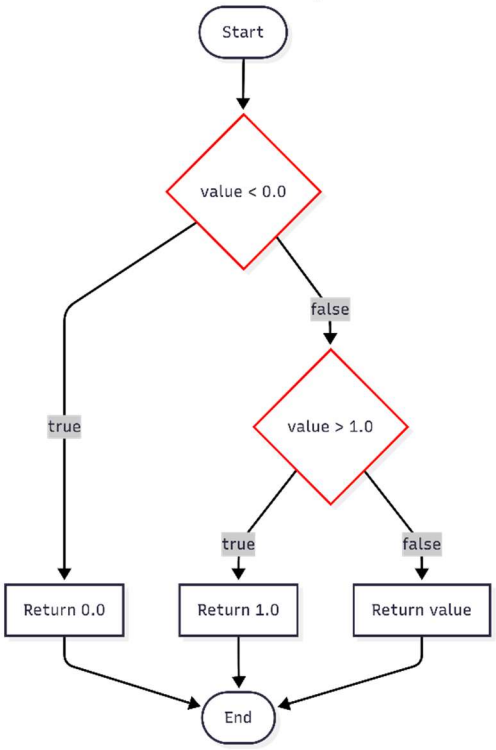
EC2: Invalid Flat Type	flatType = "10-ROOM", towns = ["BISHAN"]	Throws IllegalArgumentException ("Invalid flat type...")	`Test: EC2 (Invalid Flat Type)`
			`- Inputs:`
			`* Flat Type: '10-ROOM'`
			`* Towns: ['BISHAN']`
			`- Expected:`
			`IllegalArgumentException (Invalid flat type)`
			`- Actual:`
			`IllegalArgumentException`
			`(Invalid flat type: 10-ROOM. Valid options are: [1 Room, 2 Room, 3 Room, 4 Room, 5 Room, Executive, Multi-generation])`

EC3: Invalid Town	flatType = "4-ROOM", towns = ["GOTHAM"]	Throws IllegalArgumentException ("Invalid town...")	`Test: EC3 (Invalid Town)`
			`- Inputs:`
			`* Flat Type: '4 Room'`
			`* Towns: ['Gotham']`
			`- Expected:`
			`IllegalArgumentException (Invalid town)`
			`- Actual:`
			`IllegalArgumentException`
			`(Invalid town: Gotham. Valid towns are: [Bukit Batok, Bukit Panjang, Bukit Timah, Choa Chu Kang, Clementi, Jurong East, Jurong West, Sembawang, Woodlands, Yishun, Bukit Merah, Central Area, Geylang, Kallang/Whampoa, Queenstown, Toa Payoh, Ang Mo Kio, Bishan, Hougang, Punggol, Sengkang, Serangoon, Bedok, Marine Parade, Pasir Ris, Tampines])`

2. White Box Testing

2.1 White Box Testing 1

2.1.1 Control Flow Graph



2.1.2 Basis Paths

Cyclomatic Complexity Calculation:

- Nodes (N) = 7
- Edges (E) = 8
- CC = E - N + 2
- CC = 8 - 7 + 2 = 3

Because CC = 3, we must identify exactly 3 basis paths to achieve 100% basis path coverage

The Basis Paths:

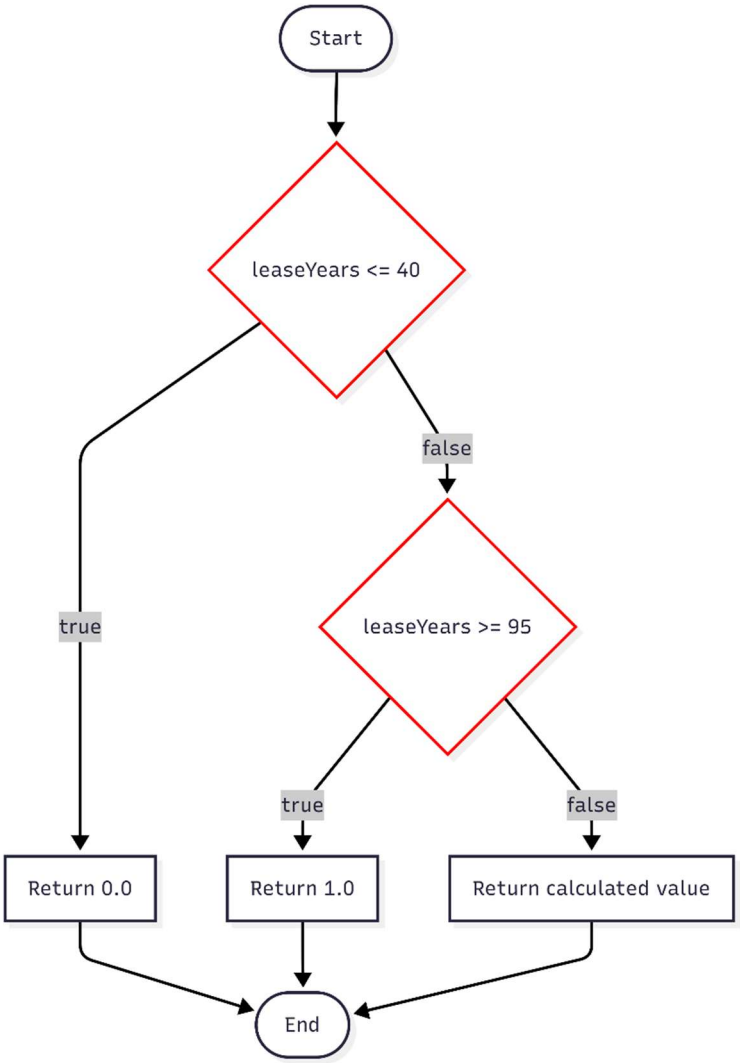
1. **Path 1:** value < 0.0 is True -> Returns 0.0
2. **Path 2:** value < 0.0 is False -> value > 1.0 is True -> Returns 1.0
3. **Path 3:** value < 0.0 is False -> value > 1.0 is False -> Returns value

Table 3: White Box Testing 1

Test Case	Test Input (Value)	Expected Output	Actual Output
Path 1 (Negative)	-0.5	0.0	Path 1 (Negative Input) - Input: -0.5 - Expected: 0.0 - Actual: 0.0
Path 2 (Over max)	1.5	1.0	Path 2 (Over Max Input) - Input: 1.5 - Expected: 1.0 - Actual: 1.0
Path 3 (Within bounds)	0.5	0.5	Path 3 (Within Bounds) - Input: 0.5 - Expected: 0.5 - Actual: 0.5

2.2 White Box Testing 2

2.2.1 Control Flow Graphs



2.2.1 Basis Paths

The Basis Paths:

1. **Path 1:** leaseYears <= 40 is True -> Returns 0.0
2. **Path 2:** leaseYears <= 40 is False -> leaseYears >= 95 is True -> Returns 1.0
3. **Path 3:** leaseYears <= 40 is False -> leaseYears >= 95 is False -> Returns calculated value

Table 4: White Box Testing 2

Test Case	Test Input (Value)	Expected Output	Actual Output
Path 1 (Low Lease)	30	0.0	- Actual: 0.0
Path 2 (High Lease)	99	1.0	- Actual: 1.0
Path 3 (Within bounds)	60	~0.363	- Actual: 0.3636363636363636